🥗 Smart Food Diet System - Project Documentation

1. Executive Summary

The **Smart Food Diet System** is a state-of-the-art health and nutrition platform that leverages **Artificial Intelligence (Google Gemini)** to provide personalized dietary guidance. The system analyzes user metrics (height, weight, age, activity level), geographic location (weather), and health goals to generate hyper-personalized meal plans and nutritional insights.

---

2. Technology Stack

Backend (The Neural Engine)

- **Framework**: FastAPI (High-performance Async Python)
- **Database**: MongoDB (via Motor Async Driver)
- **Security**: JWT (JSON Web Tokens) for authentication, Password Hashing with Bcrypt.
- **AI Integration**: Google Generative AI (Gemini Flash 1.5)
- **Environment**: Python 3.10+

Frontend (The User Interface)

- **Framework**: React 19 (Vite-powered)
- **Styling**: Tailwind CSS for modern, utility-first design.
- **Animations**: Framer Motion for smooth, premium transitions.
- **Icons**: Lucide React for consistent, high-quality iconography.
- **Data Visualization**: Recharts for interactive health analytics.

DevOps & Infrastructure

- **Containerization**: Docker & Docker Compose
- **Web Server**: Nginx (serving optimized frontend assets)
- **API Proxying**: Nginx configured to route requests to the FastAPI backend.

---

3. Core Features

🤖 AI Personal Nutritionist

Uses Google Gemini to generate:

- **Dynamic Meal Plans**: Breakfast, Lunch, Dinner, and Snacks based on user BMI and goals.
- **Personalized Insights**: 2-sentence summaries explaining why a meal plan fits the current weather and user health.
- **Natural Language Chat**: A dedicated health assistant for answering diet-related questions.

📊 Health & Macro Calculator

- **BMI Calculation**: Automatic Body Mass Index scoring.
- **Target Macros**: Dynamic calculation of Protein, Carb, and Fat targets based on user goals (Weight Loss, Muscle Gain, or Maintenance).

🌦️ Weather-Aware Nutrition

- Integrates with OpenWeather API (mapped via service) to adjust hydration and meal suggestions based on the user's local climate.

🛡️ Admin Command Center

- **User Management**: Complete registry of all system users.
- **System Health**: Real-time monitoring of database status and system load.
- **Stats Distribution**: Visual charts showing the breakdown of user goals across the platform.

---

4. File Structure & Manifest

Backend Repository (`/backend`)

- `main.py` : Entry point, CORS configuration, and router aggregation.
- `routers/` :
  - `auth.py` : Login, Registration, and JWT management.
  - `user.py` : Profile updates and user-specific data.
  - `ai.py` : Gemini-powered suggestion endpoints.
  - `recommendations.py` : Meal plan generation logic.
  - `health_tips.py` : Daily insights and weather-based tips.
- `services/` :
  - `ai_service.py` : **The Gemini Core**. Handles prompts and JSON parsing from LLM.
  - `calculator_service.py` : Pure logic for BMI and macro targets.
  - `nutrition_service.py` : Database-backed food and macro validation.
- `models/` : Pydantic schemas for data validation.
- `database/` : MongoDB connection and session management.

Frontend Repository (`/frontend`)

- `src/pages/` :
  - `Login.jsx` & `Register.jsx` : Premium split-screen authentication with hero imagery.
  - `Dashboard.jsx` : Main user hub with health widgets.
  - `AdminDashboard.jsx` : Glassmorphism-styled analytics for system admins.
  - `Profile.jsx` : User metrics management.
- `src/context/` : `AuthContext.jsx` for global state management.
- `src/api/` : `client.js` - Axios instance configured for the Docker environment.
- `public/` : Assets including `auth-hero.jpg` and `admin-hero.jpg` .

---

5. Library Dependencies

Top Backend Libraries

- `fastapi` : API Framework
- `uvicorn` : ASGI Server
- `motor` : Async MongoDB driver
- `pydantic` : Data validation
- `google-generativeai` : Gemini LLM Client
- `python-jose` : JWT handling

Top Frontend Libraries

- `react/react-dom` : UI core
- `framer-motion` : Animations
- `lucide-react` : Icon set

- `recharts` : Professional charting
- `axios` : API communication
- `tailwindcss` : Design system

---

6. API Inventory (Key Endpoints)

Method	Endpoint	Description
POST	`/api/auth/register`	Create new user with health metrics
POST	`/api/auth/login`	Secure JWT authentication
GET	`/api/users/me`	Fetch current user profile
POST	`/api/users/update-metrics`	Update weight/height/goals
GET	`/api/recommendations/meal-plan`	Generate AI/Static meal plan
GET	`/api/users/admin/stats`	Aggregate system metrics (Admin Only)
DELETE	`/api/users/admin/users/{id}`	Remove user from registry (Admin Only)

---

7. Gemini LLM Prompt Engineering

The system uses **Structured Prompting** to ensure the AI behaves as a reliable nutritionist.

- **Context injection**: User BMI, Age, and Local Weather are injected into every prompt.
- **Cultural relevance**: Prompts instruct Gemini to prefer local, familiar meal names (e.g., Indian style) over generic health foods.
- **JSON Enforcement**: The system forces Gemini to return strictly formatted JSON, which the backend then validates to ensure macros sum up correctly.

Documentation generated on 2026-03-21 • Powered by Antigravity AI Engine